

Assignment 2 DESIGN.pdf:

Description of Program:

This program implements a small number of mathematical functions (ex and px) and uses them to compute the fundamental constants e and π . It also includes a dedicated test harness named mathlib-test that compares implemented functions with that of the math library's. There is also a writeup pdf that presents the analysis of the results.

Files to be included in directory "asgn2" + pseudocode/structure:

- viete.c
 - This contains the implementation of Viète's formula to approximate π and the function to return the number of computed factors.
 - int numterms
 - double numerator
 - double prev_value
 - double current_value
 - while absolute value of (current val - prev val) is less than epsilon
 - set prev val to current val
 - set numerator to sqrt of 2 + numerator
 - set current value to prev value times (numerator/2)
 - increment numterms
 - return current value
 - return numterms
- e.c

- This contains the implementation of the Taylor series to approximate Euler's number e and the function to return the number of computed terms.
- Initialize prev value, current val, k_factorial, k, numterms
 - while absolute value of (current val - prev val) is less than epsilon
 - if k is 0
 - Set k factorial to 1
 - Increment k
 - If k is not 0
 - set prev val to current val
 - set k factorial to k factorial times k
 - set current value to current value + (1 / kfactorial)
 - Increment k
 - Increment numterms
- Return current value

- euler.c

- This contains the implementation of Euler's solution used to approximate π and the function to return the number of computed terms.
- Initialize prev value, current val, numterms, k
 - while absolute value of (current val - prev val) is less than epsilon
 - set prev val to current val
 - set current value to current value + (k * k)
 - Increment k
 - Increment numterms
 - return sqrt of 6 times current val

Return num terms

- madhava.c
 - This contains the implementation of the Madhava series to approximate π and the function to return the number of computed terms.
- Initialize current val, k, prevval, numerator, denominator, numfraction, numterms
- while absolute value of (current val - prev val) is less than epsilon
- If k is 0
- num =1
- Increment k
- Else
- Set prev value to current val
- Set num to num times 3
- Set num frac to 1 divided by num
- Set denom to 2 times k + 1
- Set current value to current val + num frac / denominator
- Increment k
- Increment num terms
- Return sqrt(12) times current val
- Return num terms
- mathlib-test.c
 - This contains the main() function which tests each of your math library functions.
 - Use getopt and switch statements to run according functions
 - ./mathlib-test [-aebmrnvsh]

- -a Runs all tests.
- -e Runs e test.
- -b Runs BBP pi test.
- -m Runs Madhava pi test.
- -r Runs Euler pi test.
- -v Runs Viete pi test.
- -n Runs Newton square root tests.
- -s Print verbose statistics.
- -h Display program synopsis and usage.
- mathlib.h
 - This contains the interface for math library
- newton.c
 - This contains the implementation of the square root approximation using Newton's method and the function to return the number of computed iterations.
 - code given on pdf
- bbp.c
 - This contains the implementation of the Bailey-Borwein-Plouffe formula to approximate π and the function to return the number of computed terms.

Initialize bbp coef, bbp frac, prev value, current val, k, num terms

while absolute value of (current val - prev val) is less than epsilon

set prev value to current val

If k is 0

Bbp coef = 1

Else

Bbp coef = bbp cpef /16

Bbp frac equals $(k(120k+151)+47) / (k(k(512k+1024) + 712) + 194) + 15)$

Current val equals current val + (bbpcoef * bbpfraction)

Increment k

Increment numterms

Return current val

Return numterms

- WRITEUP.pdf
 - This document must be a proper PDF.
 - Graphs displaying the difference between the values reported by implemented functions and that of the math library's.
 - Use a UNIX tool to produce graphs. gnuplot is recommended.
- Makefile
 - CC = clang must be specified.
 - CFLAGS = -Wall -Wextra -Werror -Wpedantic must be specified.
 - make must build the Mathlib-test executable, as should make all and make mathlib-test
 - make clean must remove all files that are compiler generated.
 - make format should format all your source code, including the header files.
- README.md
 - In proper Markdown syntax.
 - Describe how to use your program and Makefile
 - Lists and explains any command-line options that program accepts.
 - Reports any false positives by scan-build
 - Notes down any known bugs or errors in this file as well for the graders.
- DESIGN.pdf

- This document must be a proper PDF.
- Describes design and design process for program with enough detail such that a sufficiently knowledgeable programmer would be able to replicate implementation.